Ledgerly — Comprehensive Engineering & DBMS Viva Report

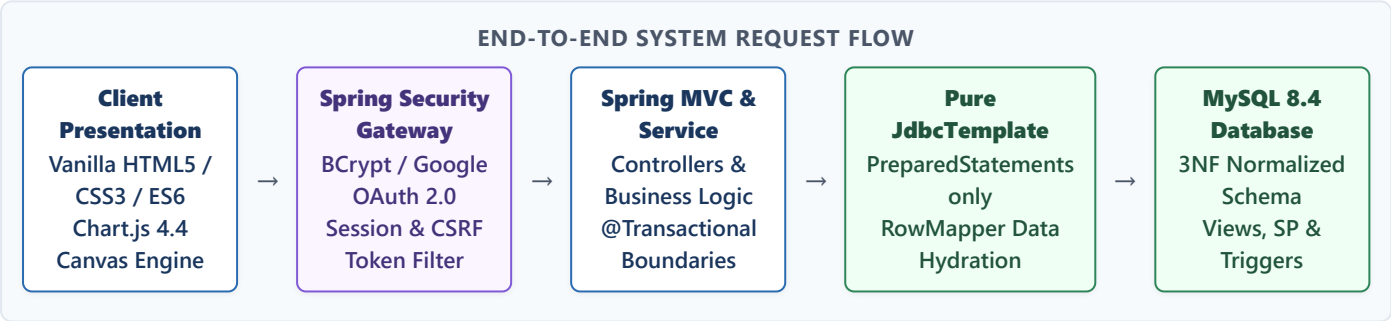
Complete Technical Guide: Java 17 Spring Boot Backend Architecture & MySQL 8 Relational Database System

BACKEND ARCHITECTURE	DATABASE ENGINE	AUTHENTICATION	LIVE PRODUCTION URL
Spring Boot 3.3 & JdbcTemplate	MySQL 8.4 (InnoDB, Strict 3NF)	BCrypt + Google OAuth 2.0 OIDC	ledgerly-b9up.onrender.com

1. Executive Project Summary

Ledgerly is a full-stack, double-entry inspired financial ledger web application engineered to demonstrate production-grade system architecture across two core computing disciplines: **Advanced Java Backend Engineering** and **Database Management Systems (DBMS)**.

Unlike conventional applications that rely on high-level Object-Relational Mapping (ORM) frameworks like Hibernate/JPA, Ledgerly was deliberately designed using **Spring JdbcTemplate with 100% hand-crafted, parameterized SQL**. This architecture ensures complete transparency, deterministic control over query execution plans, and zero hidden ORM abstractions—allowing every database transaction, stored procedure, view, and trigger to be fully explained and defended during technical evaluation.



2. Part I: Java Backend Architecture & Engineering

2.1 Framework & Core Technical Stack

Technology Component	Version & Library	Architectural Responsibility
Language Runtime	Java 17 LTS / OpenJDK	Modern language features: strong typing, records, stream pipelines, and enhanced switch expressions.
Core Framework	Spring Boot 3.3.5	Inversion of Control (IoC), Dependency Injection (DI), embedded Tomcat container, and auto-configuration.
Data Access Layer	Spring JDBC (`JdbcTemplate`)	Low-level JDBC abstraction executing raw SQL via `PreparedStatement` without ORM overhead.
Security Engine	Spring Security 6.3	Authentication filter chains, authorization gates, BCrypt password hashing, and OAuth 2.0 OIDC login.
Connection Pool	HikariCP 5.1	High-performance, zero-overhead JDBC connection pooling configured for cloud database latency.
Build & Packaging	Apache Maven 3.9	Dependency lifecycle management and multi-stage containerized compilation.

2.2 Layered Architectural Pattern (Separation of Concerns)

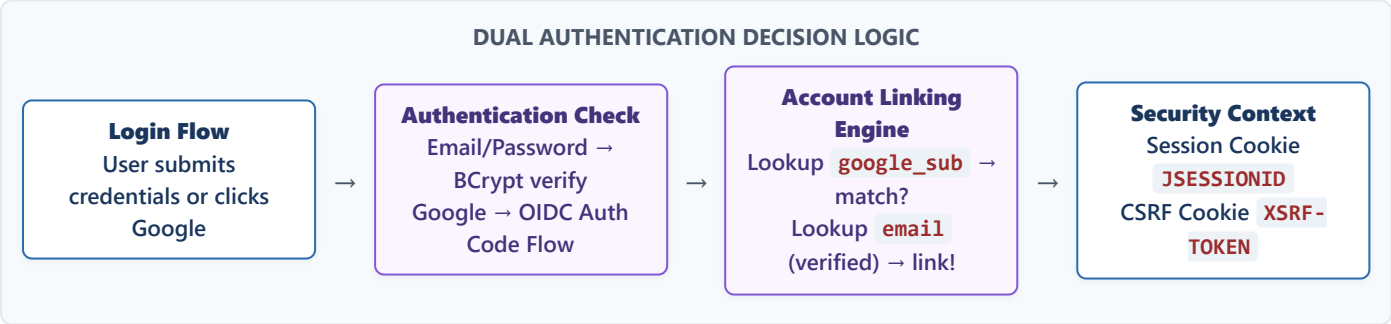
The application adheres strictly to the enterprise **Three-Tier Layered Architecture**, enforcing loose coupling and single responsibility across all packages:

- **Controller Layer (`com.ledgertracker.controller`)**: RESTful API entry points annotated with `@RestController`. Responsible for validating incoming HTTP request bodies (using `@Valid` and Jakarta Bean Validation), reading session authentication state via `@AuthenticationPrincipal`, and serializing DTO responses with appropriate HTTP status codes (`200 OK` , `201 Created` , `400 Bad Request` , `404 Not Found`).
- **Service Layer (`com.ledgertracker.service`)**: Contains domain business logic. Manages declarative transaction boundaries via `@Transactional` , performs envelope budget calculations, handles RFC 4180 CSV generation, and orchestrates account linking rules.
- **Repository Layer (`com.ledgertracker.repository`)**: Direct database access interface. Implements data retrieval and mutations using `JdbcTemplate.query()` , `queryForObject()` , and `update()` with custom functional `RowMapper<T>` lambdas to map SQL `ResultSet` rows into immutable Java domain models.
- **Model & DTO Layer (`com.ledgertracker.model` & `com.ledgertracker.dto`)**: Cleanly isolates persistent database models (e.g., `User` , `Transaction` , `Budget`) from transfer objects (e.g., `LoginRequest` , `RegisterRequest` , `TransactionRequest`). Ensures sensitive data such as password hashes or internal IDs are never leaked to the client.
- **Exception Handling Layer (`com.ledgertracker.exception`)**: A centralized `@RestControllerAdvice` class (`GlobalExceptionHandler`) intercepts all uncaught exceptions, translating validation failures (`MethodArgumentNotValidException`), business violations (`IllegalArgumentException`), and data conflicts into standardized JSON error objects:

```
{ "timestamp": "2026-10-11T...", "status": 400, "error": "Bad Request", "message": "Amount must be greater than z
```

2.3 Dual Authentication Architecture (Spring Security 6)

Ledgerly implements a session-based dual authentication scheme supporting both independent credential authentication and federated identity:



1. Local Email + Password Authentication

- **Password Hashing**: Passwords are salted and hashed using `BCryptPasswordEncoder` with a work factor of 10. Raw passwords are never logged, stored, or transmitted in responses.
- **Authentication Flow**: `AuthService.authenticate()` retrieves the user record by email, executes `passwordEncoder.matches(rawPassword, user.getPasswordHash())` , and stores a `UsernamePasswordAuthenticationToken` into Spring's `SecurityContextHolder` .

2. Federated Google OAuth 2.0 / OpenID Connect (OIDC) Flow

- **Grant Type**: Authorization Code Grant with PKCE handled natively by `spring-boot-starter-oauth2-client` .
- **Redirect URI Handling**: Configured for Cloudflare reverse proxy on Render via `server.forward-headers-strategy=framework` , guaranteeing that the generated redirect URI uses `https://` rather than internal container `http://` .
- **Account Linking Algorithm (Zero Duplicate Accounts)**:
  1. Upon successful Google login, Spring Security provides the user's OIDC payload containing `sub` (Google's immutable unique ID), `email` , `email_verified` , and `name` .

2. **Step 1 (Fast Sub Lookup)**: Query `users` by `google_sub`. If found, proceed to session issuance immediately.
3. **Step 2 (Verified Email Linking)**: If not found, inspect `email_verified`. If `true`, query `users` by `email`. If an account already exists (created via email/password), the system updates the record:

   ```
   UPDATE users SET google_sub = ?, auth_provider = 'BOTH', avatar_url = ? WHERE email = ?;
   ```

   This seamlessly binds the Google identity to the existing account without losing historical ledger records.
4. **Step 3 (New User Registration)**: If no email matches, automatically insert a new user with `password_hash = NULL`, `auth_provider = 'GOOGLE'`, and seed 10 default starting categories within a single database transaction.

> **★ Viva Defense Point: Why Pure JdbcTemplate instead of Hibernate/Spring Data JPA?**
> *"Hibernate automates query creation, but often produces severe N+1 query execution problems, unwanted full-table joins, and unpredictable SQL that cannot be analyzed in a DBMS viva. With `JdbcTemplate`, every single SQL statement is written by hand, predictable, optimized with database indexes, and protected against SQL injection using parameterized `PreparedStatement`."*

3. Part II: Database Management Systems (DBMS) & SQL Architecture

3.1 Relational Normalization Analysis (Proof of 3NF)

The database schema is formally normalized into **Third Normal Form (3NF)** to eliminate insertion, update, and deletion anomalies while ensuring minimal data redundancy:

Normal Form	Theoretical Definition	Implementation in Ledgerly
First Normal Form (1NF)	All attribute values must be atomic (indivisible). No repeating groups or composite columns.	Monetary values are discrete `DECIMAL(12,2)`; dates are ISO `DATE`; user names and emails are separate columns; payment methods and types are atomic enumeration strings.
Second Normal Form (2NF)	Table must be in 1NF and have *no partial functional dependencies* (every non-key attribute is fully dependent on the entire primary key).	For composite candidate keys (e.g. in `budgets`, where candidate key is `(user_id, category_id, month, year)`), the budget `amount` depends on the *entire combination* of user, category, and period, not any partial subset.
Third Normal Form (3NF)	Table must be in 2NF and have *no transitive functional dependencies* (non-key attributes depend only on the primary key: X → Y → Z is prohibited).	Category visual attributes (name, color, icon) are decoupled into the `categories` table. In `transactions`, only foreign key `category_id` is stored. Renaming a category updates **exactly 1 row** in `categories`, eliminating millions of redundant row modifications across transactions.

3.2 Complete Relational Schema & Integrity Constraints

```sql
-- 1. USERS TABLE: Supports dual authentication with mutually exclusive integrity
CREATE TABLE users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(191) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NULL,
    auth_provider VARCHAR(20) NOT NULL DEFAULT 'LOCAL',
    google_sub VARCHAR(100) NULL UNIQUE,
    display_name VARCHAR(100) NOT NULL,
    avatar_url VARCHAR(500) NULL,
    email_verified BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT chk_user_auth_method CHECK (password_hash IS NOT NULL OR google_sub IS NOT NULL)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 2. CATEGORIES TABLE: User-defined & default financial taxonomy
CREATE TABLE categories (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NULL,
    name VARCHAR(50) NOT NULL,
    type VARCHAR(10) NOT NULL,
    color VARCHAR(10) NOT NULL DEFAULT '#1B4332',
    icon VARCHAR(50) NOT NULL DEFAULT 'tag',
    is_default BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_category_type CHECK (type IN ('EXPENSE', 'INCOME')),
    CONSTRAINT fk_categories_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT uq_user_category UNIQUE (user_id, name, type)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. TRANSACTIONS TABLE: Double-entry financial journal
CREATE TABLE transactions (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    category_id BIGINT NOT NULL,
    amount DECIMAL(12, 2) NOT NULL,
    type VARCHAR(10) NOT NULL,
    transaction_date DATE NOT NULL,
    payment_method VARCHAR(20) NOT NULL DEFAULT 'DEBIT_CARD',
    note VARCHAR(255) NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT chk_tx_amount CHECK (amount > 0),
    CONSTRAINT chk_tx_type CHECK (type IN ('EXPENSE', 'INCOME')),
    CONSTRAINT chk_tx_payment_method CHECK (payment_method IN ('CASH', 'DEBIT_CARD', 'CREDIT_CARD', 'BANK_TRANSFER',
    CONSTRAINT fk_transactions_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_transactions_category FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 4. BUDGETS TABLE: Monthly expenditure limits
CREATE TABLE budgets (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    category_id BIGINT NOT NULL,
    `month` TINYINT NOT NULL,
    `year` SMALLINT NOT NULL,
    amount DECIMAL(12, 2) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT chk_budget_month CHECK (`month` BETWEEN 1 AND 12),
    CONSTRAINT chk_budget_year CHECK (`year` >= 2000),
    CONSTRAINT chk_budget_amount CHECK (amount > 0),
    CONSTRAINT fk_budgets_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_budgets_category FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE,
    CONSTRAINT uq_user_budget_period UNIQUE (user_id, category_id, `month`, `year`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 5. AUDIT LOGS TABLE: Immutable Change Data Capture (CDC)
```

```sql
CREATE TABLE audit_logs (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    entity_type VARCHAR(30) NOT NULL,
    entity_id BIGINT NOT NULL,
    action VARCHAR(10) NOT NULL,
    details TEXT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

🔒 Critical Constraint Explanation: `ON DELETE RESTRICT` on Categories

Notice that `fk_transactions_category` uses **`ON DELETE RESTRICT`** while `fk_transactions_user` uses `ON DELETE CASCADE`. If a user deletes an entire account, their transactions cascade. But if a user attempts to delete a single *category* that has historical transactions, MySQL strictly aborts the deletion. This prevents financial journal corruption and orphan transactions!

3.3 Advanced Database Objects (Views, Stored Procedure, Triggers)

1. SQL View: `v_monthly_summary` (Powers Dashboard KPI Cards)

Computes total inflows, total outflows, net savings, and ledger count for every user by calendar month without client loops:

```sql
CREATE OR REPLACE VIEW v_monthly_summary AS
SELECT
    t.user_id,
    YEAR(t.transaction_date) AS summary_year,
    MONTH(t.transaction_date) AS summary_month,
    COALESCE(SUM(CASE WHEN t.type = 'INCOME' THEN t.amount ELSE 0 END), 0) AS total_income,
    COALESCE(SUM(CASE WHEN t.type = 'EXPENSE' THEN t.amount ELSE 0 END), 0) AS total_expense,
    (COALESCE(SUM(CASE WHEN t.type = 'INCOME' THEN t.amount ELSE 0 END), 0) -
     COALESCE(SUM(CASE WHEN t.type = 'EXPENSE' THEN t.amount ELSE 0 END), 0)) AS net_savings,
    COUNT(t.id) AS transaction_count
FROM transactions t
GROUP BY t.user_id, YEAR(t.transaction_date), MONTH(t.transaction_date);
```

2. SQL View: `v_category_spending` (Powers Spending Doughnut Chart)

Rolls up expenses by taxonomy with average ticket sizes for Chart.js visualization:

```sql
CREATE OR REPLACE VIEW v_category_spending AS
SELECT
    t.user_id, c.id AS category_id, c.name AS category_name, c.color AS category_color, c.icon AS category_icon,
    YEAR(t.transaction_date) AS spending_year, MONTH(t.transaction_date) AS spending_month,
    COALESCE(SUM(t.amount), 0) AS total_spent, COUNT(t.id) AS transaction_count,
    ROUND(AVG(t.amount), 2) AS average_amount
FROM transactions t
JOIN categories c ON t.category_id = c.id
WHERE t.type = 'EXPENSE'
GROUP BY t.user_id, c.id, c.name, c.color, c.icon, YEAR(t.transaction_date), MONTH(t.transaction_date);
```

3. Stored Procedure: `sp_calculate_budget_usage` (Envelopes & Risk Warnings)

Accepts `(p_user_id, p_year, p_month)`, aggregates expenditures against budget limits using an aggregated subquery join, and returns dynamic risk flags:

```sql
DROP PROCEDURE IF EXISTS sp_calculate_budget_usage;
DELIMITER //
CREATE PROCEDURE sp_calculate_budget_usage(
    IN p_user_id BIGINT,
    IN p_year INT,
    IN p_month INT
)
BEGIN
    SELECT
        b.id AS budget_id, b.category_id, c.name AS category_name, c.color AS category_color,
        b.amount AS budget_amount,
        COALESCE(spent.total_spent, 0) AS actual_spent,
        (b.amount - COALESCE(spent.total_spent, 0)) AS remaining_amount,
        ROUND((COALESCE(spent.total_spent, 0) / b.amount) * 100, 2) AS percentage_used,
        CASE
            WHEN COALESCE(spent.total_spent, 0) > b.amount THEN 'EXCEEDED'
            WHEN COALESCE(spent.total_spent, 0) >= (b.amount * 0.85) THEN 'WARNING'
            ELSE 'SAFE'
        END AS budget_status
    FROM budgets b
    JOIN categories c ON b.category_id = c.id
    LEFT JOIN (
        SELECT category_id, SUM(amount) AS total_spent
        FROM transactions
        WHERE user_id = p_user_id AND type = 'EXPENSE'
          AND YEAR(transaction_date) = p_year AND MONTH(transaction_date) = p_month
        GROUP BY category_id
    ) spent ON b.category_id = spent.category_id
    WHERE b.user_id = p_user_id AND b.year = p_year AND b.month = p_month
    ORDER BY percentage_used DESC;
END //
DELIMITER ;
```

4. Storage-Engine Triggers (Immutable Audit Trail)

```sql
-- AFTER INSERT Trigger: Logs every new transaction to audit_logs
CREATE TRIGGER trg_after_transaction_insert_audit
AFTER INSERT ON transactions FOR EACH ROW
BEGIN
    INSERT INTO audit_logs (user_id, entity_type, entity_id, action, details)
    VALUES (NEW.user_id, 'TRANSACTION', NEW.id, 'INSERT',
            CONCAT('Type: ', NEW.type, ', Amount: ', NEW.amount, ', Date: ', NEW.transaction_date));
END;

-- AFTER DELETE Trigger: Logs deleted transaction records
CREATE TRIGGER trg_after_transaction_delete_audit
AFTER DELETE ON transactions FOR EACH ROW
BEGIN
    INSERT INTO audit_logs (user_id, entity_type, entity_id, action, details)
    VALUES (OLD.user_id, 'TRANSACTION', OLD.id, 'DELETE',
            CONCAT('Type: ', OLD.type, ', Amount: ', OLD.amount, ', Date: ', OLD.transaction_date));
END;
```

4. Part III: Faculty Viva Defense Guide (Questions & Model Answers)

Q1: How do you prevent SQL Injection if you are not using an ORM like Hibernate?

Answer: *"We use Spring's `JdbcTemplate` strictly with parameterized `PreparedStatement` syntax. Every user input is passed as a separate parameter object into placeholder tokens (`?`). The database driver compiles the SQL query plan before binding the parameter values as literals. This makes it impossible for input strings to alter query grammar or execute malicious SQL commands."*

Q2: How is pagination implemented efficiently in your transactions search query?

Answer: *"We implement pagination at the database engine level using `LIMIT ? OFFSET ?` combined with a composite B-Tree index on `(user_id, transaction_date DESC)`. Instead of pulling all records into Java memory and slicing the list (which would cause out-of-memory errors on large datasets), MySQL fetches only the requested page of 15 records directly from the leaf pages of the index."*

Q3: Why did you use DECIMAL(12,2) instead of FLOAT or DOUBLE for monetary amounts?

Answer: *"Floating-point types (`FLOAT`, `DOUBLE`) use IEEE 754 binary approximations, leading to rounding inaccuracies (e.g. $0.1 + $0.2 eq $0.3). In financial accounting and banking systems, monetary amounts must be exact. `DECIMAL(12,2)` is stored as an exact packed decimal representation with fixed precision and scale, guaranteeing zero rounding errors."*

Q4: Why use a Stored Procedure for budget calculations instead of computing it in Java?

Answer: *"Using the stored procedure `sp_calculate_budget_usage` provides two major advantages: first, **network minimization**—instead of transferring thousands of raw transaction rows over the network to the Java application to calculate sums, the database server aggregates the data internally and returns only the final summary rows. Second, **pre-compilation**—the execution plan is stored and cached in the MySQL procedural dictionary."*

Q5: What is the purpose of database triggers in this architecture?

Answer: *"The triggers `trg_after_transaction_insert_audit` and `trg_after_transaction_delete_audit` enforce **storage-level Change Data Capture (CDC)**. If an administrator, DBA, or migration script modifies the `transactions` table directly using a tool like MySQL Workbench or CLI (bypassing the Java web application), the trigger still fires automatically, guaranteeing an immutable audit log in `audit_logs`."*

Q6: How does your application link Google accounts with existing password accounts?

Answer: *"When a user signs in with Google, our `AuthService` checks if Google reports `email_verified = true` to prevent identity spoofing. If verified, it checks if the email already exists in our `users` table. If found, instead of throwing an error or creating a duplicate user, it updates the record with Google's unique subject identifier (`google_sub`) and sets `auth_provider = 'BOTH'`. The user can subsequently sign in using either credential."*

Q7: How is Cross-Site Request Forgery (CSRF) mitigated in your Single Page Application?

Answer: *"We configured Spring Security's `CookieCsrfTokenRepository` with a custom `SpaCsrfTokenFilter`. The server generates a cryptographic CSRF token stored in a client-readable cookie named `XSRF-TOKEN`. Our vanilla JavaScript frontend automatically reads this cookie and includes it in the `X-XSRF-TOKEN` header on every state-mutating request (`POST`, `PUT`, `DELETE`), thwarting cross-origin attacks."*

5. Part IV: Live Demonstration & Evaluation Reference

Item	Access / Configuration Detail
Live Web Application	`https://ledgerly-b9up.onrender.com` (Deployed on Render Docker)
Database Host	`mysql-2d579d8f-bhuvankeerthan-3344.c.aivencloud.com:25448` (Aiven Cloud MySQL 8.4)
Default Evaluator Login	**Email:** `elena.vance@ledgerly.internal` | **Password:** `Ledger2026!`
Google OAuth Sign-In	One-click sign-in via **"Continue with Google"** button on login page
GitHub Repository	`https://github.com/bhuvankrithik-gth/expense-tracker`